

Bài 1:

Chỉnh sửa chương trình để đếm số lượng cặp ngoặc tròn hợp lệ trong chuỗi.
Ví dụ: `((()))` → 3 cặp.

```
#include <stdio.h>
#include <string.h>

int main() {
    char s[100];
    int i, count = 0, cap = 0;

    printf("Nhap xau: ");
    scanf("%s", s);

    for (i = 0; i < strlen(s); i++) {
        if (s[i] == '(')
            count++;
        else if (s[i] == ')') {
            if (count > 0) {
                count--;
                cap++;
            }
        }
    }

    printf("So cap ngoac hop le: %d\n", cap);
    return 0;
}
```

Ví dụ:

less

 Sao chép mã

Nhập xâu: `((()))`

→ Số cặp ngoặc hợp lệ: 3

Giải thích:

Ba cặp hợp lệ là:

- (1,2), (3,6), (4,5)

Bài 2:

Viết chương trình kiểm tra **ngoặc tròn**, **ngoặc vuông** và **ngoặc nhọn** đều hợp lệ.

Ví dụ: { [() ()] } → hợp lệ.

```
#include <stdio.h>
#include <string.h>

char stack[100];
int top = -1;

void push(char c) { stack[++top] = c; }
char pop() { return (top == -1) ? '\0' : stack[top--]; }

int main() {
    char s[100];
    int i;
    printf("Nhập xâu: ");
    scanf("%s", s);

    for (i = 0; i < strlen(s); i++) {
        char c = s[i];
        if (c == '(' || c == '[' || c == '{')
            push(c);
        else if (c == ')' || c == ']' || c == '}') {
            char t = pop();
            if ((c == ')' && t != '(') ||
                (c == ']' && t != '[') ||
                (c == '}' && t != '{')) {
                printf("No\n");
                return 0;
            }
        }
    }

    if (top == -1)
        printf("Yes\n");
    else
        printf("No\n");
    return 0;
}
```

Ví dụ 1:

CSS

 Sao chép mã

Nhập xâu: {[()()]}

→ Yes

Ví dụ 2:

CSS

 Sao chép mã

Nhập xâu: {[()]}

→ No

Giải thích:

Trường hợp thứ hai sai vì ngoặc [mở ra nhưng bị đóng bởi).

Bài 3:

Thay vì nhập chuỗi qua `scanf`, hãy **nhập chuỗi có khoảng trắng** (dùng `gets()` hoặc `fgets()`).

```
#include <stdio.h>
#include <string.h>

int main() {
    char s[100];
    int i, count = 0;

    printf("Nhập xâu: ");
    fgets(s, sizeof(s), stdin); // Đọc cả khoảng trắng

    for (i = 0; i < strlen(s); i++) {
        if (s[i] == '(')
            count++;
        else if (s[i] == ')')
            count--;

        if (count < 0) {
            printf("No\n");
            return 0;
        }
    }

    printf(count == 0 ? "Yes\n" : "No\n");
    return 0;
}
```

Ví dụ:

CSS

 Sao chép mã

Nhập xâu: ((a + b) * (c + d))

→ Yes

CSS

 Sao chép mã

Nhập xâu: (a + b)) * (c + d)

→ No

Giải thích:

Sử dụng `fgets()` để đọc được cả dấu cách.

Bài 4:

Viết chương trình cho phép **nhập nhiều dòng** (tối đa 5 dòng), mỗi dòng là một biểu thức, và **in ra kết quả hợp lệ / không hợp lệ** cho từng dòng.

```
#include <stdio.h>
#include <string.h>

int check(char s[]) {
    int count = 0;
    for (int i = 0; i < strlen(s); i++) {
        if (s[i] == '(') count++;
        else if (s[i] == ')') count--;
        if (count < 0) return 0;
    }
    return count == 0;
}

int main() {
    char s[100];
    for (int i = 1; i <= 5; i++) {
        printf("Dong %d: ", i);
        scanf("%s", s);
        printf(check(s) ? "Yes\n" : "No\n");
    }
    return 0;
}
```

Ví dụ:

yaml

Sao chép mã

Dong 1: (())

→ Yes

Dong 2: (() ()

→ No

Dong 3: ()

→ Yes

Dong 4:)(

→ No

Dong 5: ((()))

→ Yes

Bài 5:

Thêm chức năng **đếm số ngoặc sai vị trí** (ví dụ: `() ) (` → sai 2 ngoặc).

```
#include <stdio.h>
#include <string.h>

int main() {
    char s[100];
    int i, count = 0, sai = 0;

    printf("Nhap xau: ");
    scanf("%s", s);

    for (i = 0; i < strlen(s); i++) {
        if (s[i] == '(')
            count++;
        else if (s[i] == ')') {
            if (count > 0)
                count--;
            else
                sai++; // có ngoặc đóng sai vị trí
        }
    }

    sai += count; // số ngoặc mở chưa đóng
    printf("So ngoac sai: %d\n", sai);
    return 0;
}
```

Ví dụ 1:

bash

 Sao chép mã

Nhập xâu: `((()))`

→ Số ngoặc sai: 1

Ví dụ 2:

less

 Sao chép mã

Nhập xâu: `)()(`

→ Số ngoặc sai: 2

Giải thích:

- Trong `)()(`, có 1 ngoặc `)` sai đầu tiên và 1 ngoặc `(` chưa đóng → tổng 2 sai.

Bài 6:

Chuyển chương trình thành **hàm kiểm tra**:

```
int kiemTraNgoac(char s[]);
```

và viết `main()` gọi hàm này nhiều lần.

```
#include <stdio.h>
#include <string.h>

int kiemTraNgoac(char s[]) {
    int count = 0;
    for (int i = 0; i < strlen(s); i++) {
        if (s[i] == '(') count++;
        else if (s[i] == ')') count--;
        if (count < 0) return 0;
    }
    return count == 0;
}

int main() {
    char s[100];
    printf("Nhap xau: ");
    scanf("%s", s);
    printf(kiemTraNgoac(s) ? "Yes\n" : "No\n");
    return 0;
}
```

Ví dụ:

bash

 Sao chép mã

Nhập xâu: ((()))

→ Yes

css

 Sao chép mã

Nhập xâu: (() (

→ No

Giải thích:

Hàm `kiemTraNgoac()` trả về `1` nếu hợp lệ, `0` nếu không.

Bài 7:

Viết chương trình **tự động sinh ra 10 chuỗi ngẫu nhiên** chỉ chứa (và) rồi in ra chuỗi nào hợp lệ.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <string.h>

int kiemTra(char s[]) {
    int c = 0;
    for (int i = 0; i < strlen(s); i++) {
        if (s[i] == '(') c++;
        else c--;
        if (c < 0) return 0;
    }
    return c == 0;
}

int main() {
    srand(time(NULL));
    for (int t = 0; t < 10; t++) {
        char s[21];
        int len = rand() % 10 + 5;
        for (int i = 0; i < len; i++)
            s[i] = rand() % 2 ? '(' : ')';
        s[len] = '\0';
        printf("%s → %s\n", s, kiemTra(s) ? "Yes" : "No");
    }
    return 0;
}
```

Ví dụ (chạy mẫu):

scss

 Sao chép mã

```
((()))( → No
()()()() → Yes
((( ))) → Yes
)))(( → No
()()()() → Yes
()(( ))( → No
((( ))) → Yes
(( ))(( → No
()() → Yes
))(( → No
```

Giải thích:

Mỗi lần chạy chương trình, chuỗi ngẫu nhiên khác nhau.

Bài 8:

Thêm chức năng **thông báo vị trí sai đầu tiên** (chỉ số của ngoặc làm chuỗi không hợp lệ).

```
#include <stdio.h>
#include <string.h>

int main() {
    char s[100];
    int count = 0;

    printf("Nhap xau: ");
    scanf("%s", s);

    for (int i = 0; i < strlen(s); i++) {
        if (s[i] == '(')
            count++;
        else if (s[i] == ')')
            count--;
        if (count < 0) {
            printf("Sai tai vi tri: %d\n", i);
            return 0;
        }
    }

    if (count == 0)
        printf("Hop le\n");
    else
        printf("Thieu %d dau ')' o cuoi\n", count);
    return 0;
}
```

Ví dụ 1:

bash

 Sao chép mã

Nhập xâu: ())(

→ Sai tại vị trí: 2

Giải thích:

Ký tự `)` ở vị trí 2 (đếm từ 0) khiến `count` âm.

Ví dụ 2:

less

 Sao chép mã

Nhập xâu: ((()))

→ Hợp lệ

Bài 9:

Viết lại chương trình **không dùng** `strlen()`, mà duyệt chuỗi cho đến khi gặp ký tự `'\0'`.

```
#include <stdio.h>

int main() {
    char s[100];
    int count = 0, i = 0;

    printf("Nhap xau: ");
    scanf("%s", s);

    while (s[i] != '\0') {
        if (s[i] == '(')
            count++;
        else if (s[i] == ')')
            count--;
        if (count < 0) {
            printf("No\n");
            return 0;
        }
        i++;
    }

    printf(count == 0 ? "Yes\n" : "No\n");
    return 0;
}
```

Ví dụ:

bash

 Sao chép mã

Nhập xâu: (()())

→ Yes

CSS

 Sao chép mã

Nhập xâu: (() (

→ No

Giải thích:

Không dùng `strlen()`, mà duyệt ký tự đến khi gặp ký tự kết thúc chuỗi.

Bài 10:

Viết chương trình **kiểm tra biểu thức toán học** có ngoặc hợp lệ hay không, ví dụ:

$(2 + 3) * (4 - 5) \rightarrow \text{Yes}$

$((3+2)*5 \rightarrow \text{No}$

```
#include <stdio.h>
#include <string.h>

int main() {
    char s[200];
    int count = 0;

    printf("Nhap bieu thuc: ");
    fgets(s, sizeof(s), stdin);

    for (int i = 0; s[i] != '\0'; i++) {
        if (s[i] == '(')
            count++;
        else if (s[i] == ')')
            count--;
        if (count < 0) {
            printf("No\n");
            return 0;
        }
    }

    printf(count == 0 ? "Yes\n" : "No\n");
    return 0;
}
```

Ví dụ 1:

CSS

 Sao chép mã

Nhập biểu thức: $(2 + 3) * (4 - 5)$

→ Yes

Ví dụ 2:

CSS

 Sao chép mã

Nhập biểu thức: $((3+5)*2$

→ No

Ví dụ 3:

CSS

 Sao chép mã

Nhập biểu thức: $5 + 6 * 7)$

→ No

Giải thích:

Chỉ kiểm tra việc đóng mở ngoặc, không cần phân tích toán học.